

Техническое задание (ТЗ) на разработку системы прогнозирования пассажиропотока трамваев Москвы

Версия 1.0 | 2026-09-09

1. Введение

1.1. Цель проекта

Разработать интерактивный веб-сервис для краткосрочного, среднесрочного и долгосрочного прогнозирования пассажиропотока на трамвайных маршрутах Москвы. Сервис должен обрабатывать исторические данные валидаций оплаты проезда и телематики (GPS-треки, интервалы движения) за несколько лет, строить прогнозы с привязкой к геопозиции (остановки, маршруты, административные округа) и предоставлять диспетчерам удобный дашборд для принятия решений о распределении подвижного состава.

1.2. Бизнес-ценность

- Повышение точности прогноза загруженности → оптимизация выпуска вагонов на линию.
- Снижение переполненности в часы пик и уменьшение порожнего пробега.
- Возможность планирования ремонтных работ и открытия новых маршрутов на основе долгосрочных трендов.

1.3. Область применения

- Городской транспортный комплекс Москвы (в первую очередь трамвайная сеть).
 - Потенциально масштабируемо на другие виды наземного транспорта (автобусы, электробусы).
-

2. Состав команды и роли

Участник	Роль в проекте	Основные компетенции	Зона ответственности
Максим Богуславский	Team Lead / Backend-разработчик / ML-generalist	Python, FastAPI, React, TS, прогнозирование временных рядов (опыт ODS.ai), Kafka, архитектура	Общее руководство, архитектура системы, разработка бэкенда (API, интеграция моделей), помощь фронтенду, координация, финальная демонстрация
Максим Баев	Frontend-разработчик / UI/UX-дизайнер	Геймдев, 3D-дизайн, изучение CV, опыт работы с графикой, дизайн интерфейсов	Разработка интерактивного дашборда (React + карты), дизайн макетов, визуализация данных (графики, тепловые карты, 3D-сцены при необходимости)
Павел	ML Engineer / Data Engineer	CV/ML, обработка больших данных, оптимизация, реальные проекты (Норникель)	ETL-пайплайны, разработка и обучение моделей (XGBoost, LSTM, GCN), тюнинг гиперпараметров, работа с GPU, обеспечение производительности инференса

Света	Product Analyst / QA / Координатор	Аналитика, понимание IT, тестирование, коммуникации	Анализ данных (EDA), формулировка требований к дашборду, проверка качества прогнозов, тестирование интерфейса, подготовка презентации и документации
--------------	--	--	--

3. Функциональные требования

3.1. Основные возможности системы

1. Загрузка и хранение данных

- Импорт исторических данных валидаций (оплата проезда) и телематики (GPS-координаты, временные метки, номера маршрутов, идентификаторы транспортных средств).
- Поддержка форматов CSV, Parquet, JSON (в зависимости от предоставленных данных).
- Хранение агрегированных данных в удобной для аналитики структуре (например, ClickHouse или PostgreSQL с временными таблицами).

2. Построение прогнозов (три горизонта)

- **Краткосрочный (1 день)** – почасовой или поминутный прогноз пассажиропотока на каждой остановке и маршруте в целом.
- **Среднесрочный (1 месяц)** – ежедневный прогноз с учётом сезонности, праздников, погодных факторов.
- **Долгосрочный (1 год)** – помесечный прогноз с учётом открытия новых маршрутов, изменения городской инфраструктуры (мета-обучение для новых остановок).

3. Фильтрация и агрегация прогнозов

- По конкретному маршруту, остановке, административному округу (ЦАО, ЮАО и т.д.).
- По временному интервалу (час, день, неделя, месяц).
- Возможность сравнения прогноза с фактическими данными за прошлые периоды.

4. Интерактивный дашборд

- Карта Москвы с нанесёнными трамвайными остановками и маршрутами.
- Цветовая кодировка остановок по прогнозируемой загрузке (от зелёного – низкая до красного – высокая).

- Временной слайдер для просмотра динамики изменения загрузки.
 - Графики пассажиропотока по выбранным объектам (линейные, столбчатые).
 - Возможность экспорта отчётов (PNG, PDF, CSV).
5. **API для интеграции**
- RESTful API на базе FastAPI для получения прогнозов в машиночитаемом формате (JSON).
 - Эндпоинты: `/predict/stop/{id}`, `/predict/route/{id}`, `/predict/district/{id}?horizon=day|month|year`.
 - Документация OpenAPI (Swagger) для удобства интеграции.
6. **Масштабируемость**
- Система должна обрабатывать запросы от нескольких диспетчеров одновременно.
 - Прогнозы должны пересчитываться не реже одного раза в сутки (для краткосрочного – каждый час при поступлении новых данных).

3.2. Нефункциональные требования

- **Производительность:** время отклика API на запрос прогноза – не более 2 секунд.
- **Надёжность:** система должна быть устойчива к сбоям, иметь логирование ошибок.
- **Безопасность:** базовое разграничение доступа (роли: «диспетчер», «администратор»).
- **Кроссплатформенность:** веб-сервис должен работать в современных браузерах (Chrome, Firefox, Safari).
- **Мобильная адаптация:** дашборд должен корректно отображаться на планшетах (для диспетчеров в полевых условиях).

4. Технологический стек (предварительный)

Компонент	Технология	Обоснование
Бэкенд	Python 3.10+, FastAPI, Uvicorn	Быстрый, асинхронный, лёгкий, хорошо документирован, интеграция с ML-моделями (via ONNX или TorchServe)
База данных	PostgreSQL + TimescaleDB (для временных рядов) или ClickHouse	Работа с большими объёмами временных данных, поддержка агрегаций, оконных функций

Кэш	Redis	Кэширование прогнозов для ускорения API, хранение промежуточных результатов
ML-фреймворки	PyTorch (для нейросетей), XGBoost/LightGBM, scikit-learn	Широкий спектр моделей, поддержка GPU (PyTorch) и CPU-оптимизация (бустинги)
Графовые модели	PyTorch Geometric (для GCN)	Эффективная работа с графами остановок/маршрутов
Фронтенд	React + TypeScript, Redux Toolkit (для состояния)	Популярный, масштабируемый, большой выбор библиотек для карт и графиков
Карты и визуализация	Leaflet / Mapbox GL JS (карты), D3.js (графики), React-Leaflet	Интерактивность, поддержка слоёв, возможность кастомизации
Контейнеризация	Docker, Docker Compose	Упрощение развёртывания, воспроизводимость, изоляция сервисов
Оркестрация данных	Apache Airflow (опционально)	Планирование ETL-задач и переобучения моделей
Формат модели	ONNX или TorchScript (для инференса)	Ускорение предсказаний на CPU/GPU

5. Этапы разработки и распределение задач

Проект разбивается на **4 основных этапа**, каждый из которых имеет чёткие результаты и ответственных. Рекомендуемый режим работы – параллельно-итеративный: после завершения Этапа 1 фронтенд- и бэкэнд-команды могут работать независимо.

Этап 0: Подготовка и старт (День 1, 4-6 часов)

Задача	Ответственный	Результат
Утверждение трека, финализация состава и ролей	Все	Закреплённые ответственные
Создание репозитория (GitHub/GitLab) с базовой структурой папок	Богуславский	Репозиторий с README, .gitignore, базовым docker-compose.yml
Настройка окружения для каждого участника (Python, npm, Docker)	Все	Рабочие локальные среды
Получение и первичное изучение данных (если организаторы предоставили)	Павел + Света	Отчёт по EDA (структура, пропуски, выбросы, гранулярность)
Определение ключевых метрик качества (RMSLE, MAE, MAPE) и валидационной стратегии	Богуславский + Павел	Выбор метрики (предположительно RMSLE, как в прошлом опыте) и схема кросс-валидации по времени
Создание макетов дашборда (Figma или бумажные эскизы)	Баев	Макеты ключевых экранов (карта, детальный график, фильтры)

Этап 1: Разработка пайплайна данных и базовой ML-модели (Дни 2-3, ~16 часов)

Цель: создать ETL-процесс для агрегации данных в удобный формат и обучить первую рабочую модель (бейзлайн), которая даст точку отсчёта.

Задачи для Павла (ML / Data Engineer)

1. Проектирование схемы хранения

- Определить, как будут храниться данные: временные ряды (Timestamp, stop_id, route_id, passengers, occupancy, weather, day_of_week, holiday_flag и т.д.).
- Выбрать и развернуть СУБД (рекомендуется **TimescaleDB** как расширение PostgreSQL или **ClickHouse** для аналитики).
- Написать скрипты для импорта данных из предоставленных файлов в БД (с использованием Pandas, Dask или прямой загрузки).

2. Агрегация и создание признаков

- Разбить данные по временным интервалам: час, день, месяц.
- Создать календарный справочник (праздники, выходные, школьные каникулы, дни проведения крупных мероприятий).
- Добавить погодные данные (температура, осадки, видимость) из открытых API (например, OpenWeatherMap), если не предоставлены.
- Сформировать признаки для **каждой остановки**:
 - Тип района (жилой, деловой, смешанный) – можно получить из геоданных.
 - Близость к метро, ж/д станциям, ТЦ, вузам.
 - Количество маршрутов, проходящих через остановку.
 - Исторические скользящие средние за 7, 30, 90 дней.
- Создать **граф остановок** (узлы – остановки, рёбра – если они находятся на одном маршруте или расстояние между ними меньше порога) для последующего использования в GCN.

3. Обучение базовой модели (XGBoost / LightGBM)

- Обучить модель на агрегированных дневных данных с признаками из п.2.
- Провести первичный тюнинг гиперпараметров (через Optuna или GridSearch).
- Оценить качество на отложенной выборке (последние 30 дней) – зафиксировать бейзлайн RMSLE.
- Экспортировать модель в формат ONNX для быстрого инференса.

Задачи для Богуславского (Backend / Team Lead)

1. Разработать структуру API (FastAPI)

- Спроектировать эндпоинты (см. п.3.1.5).
- Реализовать базовый эндпоинт `/predict/stop` для получения прогноза от XGBoost модели (заглушка, возвращающая среднее значение).
- Написать документацию OpenAPI.

2. Подготовка контейнеризации

- Создать Dockerfile для бэкенда.
- Написать docker-compose.yml с сервисами: backend, postgres/timescaledb, redis.

3. Организация кэширования

- Настроить Redis для кэширования прогнозов (ключ = комбинация параметров запроса, значение = прогноз с TTL).

Задачи для Баева (Frontend / UI/UX)

1. Разработка базового React-приложения

- Инициализировать проект (Create React App или Vite + TypeScript).
- Установить библиотеки: react-leaflet (карта), chart.js или recharts (графики), material-ui или antd (компоненты).

2. Создание макета дашборда

- Реализовать боковую панель с фильтрами (выбор маршрута, остановки, округа, даты).
- Вставить карту Москвы (использовать бесплатные тайлы OpenStreetMap или Mapbox).
- Добавить тестовые маркеры остановок (заглушки) с произвольными цветами.

3. Интеграция с API (заглушка)

- Написать сервис для запросов к бэкенду (fetch / axios).
- По клику на маркер показывать модальное окно с прогнозом (пока статическим).

Задачи для Светы (Аналитик / QA)

1. Формализация требований к дашборду

- Составить список сценариев использования от лица диспетчера (например: «Хочу увидеть загрузку на маршруте №10 в час пик на следующей неделе»).
- Создать User Story Map.

2. Тестирование ETL и модели

- Проверить корректность агрегации (сравнить с сырыми данными).
- Оценить бейзлайн-модель на различных временных срезах, выявить систематические ошибки (например, недооценка в выходные).

Итог Этапа 1:

- Работающая база данных с агрегированными данными.
- Обученная базовая модель XGBoost с известной точностью.
- Бэкенд с одним работающим эндпоинтом и кэшированием.
- Фронтенд с картой, маркерами и базовыми фильтрами.

Этап 2: Разработка продвинутых ML-моделей и интеграция (Дни 4-6, ~24 часа)

Цель: улучшить точность прогнозов за счёт более сложных моделей (LSTM, GCN) и гибридизации, а также расширить функциональность API и дашборда.

Задачи для Павла (ML)

1. **Реализация модели LSTM для краткосрочного прогноза**
 - Использовать опыт прошлого участия: окно = 409 дней, hidden_size = 128, lr = 0.0005.
 - Применить регуляризацию (dropout, ранняя остановка).
 - Обучить модель для каждой остановки или для группы остановок (можно сделать единую модель с эмбедами остановок).
 - Сохранить модель в формате TorchScript или ONNX.
2. **Реализация GCN + LSTM (пространственно-временная модель)**
 - Использовать PyTorch Geometric.
 - Построить граф на основе топологии маршрутов и географической близости.
 - Обучить модель, которая принимает на вход признаки всех остановок за последние L дней и выдаёт прогноз на следующий день для каждой остановки.
 - Сравнить с LSTM без графа.
3. **Среднесрочный прогноз (ансамбль бустингов)**
 - Улучшить XGBoost, добавив больше признаков (лаговые значения, погода, календарь).
 - Добавить LightGBM и CatBoost, сделать взвешенное усреднение.
 - Калибровка прогнозов с помощью коэффициента c (по опыту – около 1.04) и гибридизации с нейросетевой моделью через вес w (0.25-0.3).
4. **Долгосрочный прогноз (Марковские цепи + мета-обучение)**
 - Построить матрицы переходов между районами (или остановками) на основе исторических данных.
 - Реализовать мета-обучение для новых остановок (использовать предобученную модель и дообучать на малом количестве данных с помощью MAML или простого fine-tune).
 - Интегрировать в общий ансамбль.
5. **Оптимизация инференса**
 - Конвертировать все модели в ONNX или использовать TorchScript.
 - Настроить батчевый инференс для ускорения обработки тысяч остановок.

Задачи для Богуславского (Backend / Team Lead)

1. **Расширение API**
 - Добавить новые эндпоинты: `/predict/route`, `/predict/district`, `/predict/compare` (сравнение прогноза с фактом).
 - Реализовать параметры `horizon` (day/month/year) и `granularity` (hour/day/week).

- Поддержка фильтрации по датам.
- 2. **Интеграция моделей**
 - Создать модуль загрузки моделей (Factory pattern) – при старте сервиса загружаются все модели в память.
 - Для каждого запроса выбирать соответствующую модель в зависимости от горизонта и типа объекта.
 - Реализовать гибридизацию (w-смешивание) на уровне API.
- 3. **Потоковое обновление прогнозов**
 - Реализовать фоновый процесс (Celery или APScheduler) для пересчёта прогнозов раз в сутки или по расписанию.
 - Обновлять кэш Redis после пересчёта.
- 4. **Логирование и мониторинг**
 - Добавить логирование всех запросов и ошибок в структурированном формате (JSON).
 - Настроить базовый мониторинг (количество запросов, время ответа).

Задачи для Баева (Frontend / UI/UX)

1. **Визуализация прогнозов на карте**
 - Цветовая кодировка маркеров остановок в зависимости от прогнозируемой загрузки (5 градаций).
 - Добавить временной слайдер для просмотра загрузки в разные часы/дни.
 - Отображение маршрутов линиями (с толщиной, зависящей от загрузки).
2. **Детальная страница остановки/маршрута**
 - При клике на маркер или элемент списка открывать панель с графиком прогноза (линейный график за неделю/месяц).
 - Возможность переключения между горизонтами (день, месяц, год).
3. **Агрегация по районам/округам**
 - Выпадающий список округов, при выборе – подсветка на карте и отображение суммарного прогноза на панели.
4. **Дружественный UI**
 - Адаптация под планшеты.
 - Тёмная/светлая тема.
 - Интерактивные подсказки (tooltips) для элементов.
5. **Экспорт данных**
 - Кнопка «Скачать отчёт» – генерирует PDF или PNG текущего вида дашборда.

Задачи для Светы (Аналитик / QA)

1. **Валидация качества прогнозов**
 - Провести сравнение моделей на отложенной выборке.
 - Построить графики ошибок по времени суток, дням недели, типам районов.
 - Выявить систематические смещения и передать команде ML для калибровки.

2. Тестирование API и фронтенда

- Написать чек-лист тестовых сценариев (позитивных и негативных).
- Провести ручное функциональное тестирование.
- Составить отчёт о найденных багах.

3. Подготовка презентации

- Собрать материалы: скриншоты дашборда, графики улучшения точности, техническую архитектуру.
- Подготовить речь для защиты.

Итог Этапа 2:

- Несколько обученных моделей (LSTM, GCN+LSTM, ансамбль бустингов, марковские цепи).
 - Бэкенд со всеми эндпоинтами, интеграцией моделей и расписанием обновления прогнозов.
 - Полнофункциональный дашборд с картой, графиками, фильтрами и экспортом.
 - Предварительные результаты качества (улучшение относительно бейзлайна на 15-30%).
-

Этап 3: Финальная полировка и защита (День 7, ~8 часов)

Цель: завершить все интеграции, исправить критичные баги, оптимизировать производительность, подготовить финальную презентацию и демо.

Задачи для всех

- **Павел:** финальный тюнинг гиперпараметров, проверка стабильности моделей на новых данных (если появятся), подготовка ONNX-экспорта для инференса.
- **Богуславский:** финальная настройка деплоя (docker-compose production), проверка безопасности (CORS, rate limiting), документирование API.
- **Баев:** исправление визуальных багов, улучшение анимаций, финальная адаптация под мобильные устройства.
- **Света:** повторное полное тестирование, подготовка финальной презентации (PowerPoint / Google Slides), репетиция демо.

Финальные проверки:

- Нагрузочное тестирование API (например, 100 параллельных запросов).
 - Проверка корректности прогнозов для всех трёх горизонтов.
 - Проверка работы дашборда в браузерах (Chrome, Firefox, Safari).
-

6. Требования к данным и предобработке (детально)

6.1. Источники данных (гипотетические, уточнить у организаторов)

- **Валидации оплаты проезда:** ID транспортного средства, ID остановки, время валидации, тип билета (предположительно).
- **Телематика:** GPS-координаты, скорость, ID маршрута, ID транспортного средства, временные метки.
- **Справочники:** список остановок с координатами, список маршрутов с привязкой к остановкам, административное деление Москвы.

6.2. Предобработка

- **Очистка:** удаление дубликатов, обработка пропусков (интерполяция или заполнение средним/медианным по аналогичному времени).
- **Агрегация:** группировка по часам/дням для каждой остановки и маршрута.
- **Вычисление пассажиропотока:** сумма валидаций за интервал (предполагаем, что каждая валидация = один пассажир).
- **Нормализация:** для нейросетей – StandardScaler по каждому признаку отдельно (чтобы избежать влияния разных масштабов). Для табличных моделей – можно использовать логирование ($\log 1p$) для снижения влияния выбросов.
- **Создание графа остановок:**
 - Вершины – остановки.
 - Рёбра – если остановки находятся на одном маршруте (вес = 1) или расстояние между ними меньше 500 м (вес = $1 / \text{расстояние}$).
 - Можно добавить рёбра на основе частоты пересадок.

6.3. Признаки (Feature Engineering)

Календарные:

- День недели (one-hot или циклическое кодирование: $\sin/\cos$).
- Месяц, квартал, год.
- Праздничный флаг (да/нет), предпраздничный день, послепраздничный день.
- Номер недели в году.

Погодные:

- Температура (средняя за день/час).
- Осадки (тип, количество).
- Влажность, видимость.
- Скорость ветра.

Временные лаги:

- Значение пассажиропотока за 1, 2, 3, 7, 14, 30, 90 дней до прогнозируемой даты.
- Скользящее среднее за 7, 30 дней.
- Отношение текущего значения к среднему за аналогичный период прошлого года.

Географические/статичные:

- Тип района (жилой, деловой, смешанный) – можно получить по координатам через OpenStreetMap или из готовых справочников.
- Расстояние до ближайшей станции метро.
- Количество маршрутов, проходящих через остановку.
- Плотность населения в радиусе 500 м (если доступно).

Поведенческие (временные ряды):

- Тренд (коэффициент линейной регрессии за последние 30 дней).
- Сезонная компонента (из декомпозиции STL).

Мета-признаки для новых остановок:

- Профиль похожей остановки (см. трансферное обучение) – вычисляется как косинусное расстояние между векторами статических признаков.

7. Критерии приемки (Success Criteria)

7.1. Функциональные критерии

1. Система принимает на вход исторические данные и строит прогноз на 1 день, 1 месяц, 1 год для любой остановки, маршрута или округа.
2. Дашборд отображает карту с цветовыми маркерами, временной слайдер, графики прогнозов и фильтры.
3. Прогноз обновляется автоматически (не реже раза в сутки).
4. API предоставляет прогноз в JSON-формате с временем выполнения < 2 секунд.
5. Система выдерживает нагрузку не менее 50 одновременных пользователей.

7.2. Качественные критерии (метрики)

- **RMSLE** (Root Mean Squared Logarithmic Error) на тестовой выборке (последние 30 дней) должен быть улучшен относительно бейзлайна (историческое среднее) не менее чем на 15%.
- Для краткосрочного прогноза (1 день) целевой RMSLE < 0.45 (гипотетически, уточнить на основе данных).
- Для среднесрочного (1 месяц) RMSLE < 0.50.
- Для долгосрочного (1 год) RMSLE < 0.60.

7.3. UX/UI критерии

- Дашборд интуитивно понятен диспетчеру без дополнительного обучения.
- Переключение между вкладками и фильтрами происходит плавно (< 1 секунда).
- Карта загружается не дольше 3 секунд.
- Все графики интерактивны (при наведении показываются значения).

7.4. Технические критерии

- Все компоненты упакованы в Docker-контейнеры и запускаются одной командой `docker-compose up`.
 - Код покрыт комментариями и документацией (минимум README с инструкцией по запуску).
 - Логирование ошибок и информационных сообщений настроено.
-

8. Риски и стратегии их снижения (расширенный список)

Риск	Вероятность	Влияние	Стратегия смягчения
Данные предоставлены в неожиданном формате или с большим количеством пропусков	Средняя	Высокое	Заложить 2-3 часа на EDA и написание гибких парсеров. Использовать Pandas с резервными стратегиями заполнения.
Нехватка GPU-ресурсов у Павла для обучения LSTM/GCN	Средняя	Среднее	Использовать облачные решения (Colab, Kaggle) с бесплатным GPU или ограничиться CPU-моделями (XGBoost) для MVP.

Баев не успевает освоить React + карты за отведённое время	Низкая	Среднее	Богуславский помогает с интеграцией карт, а Баев фокусируется на дизайне (Figma) и статичных макетах, которые вёрстка реализует совместно.
Сложность интеграции нескольких моделей в единый API	Средняя	Среднее	Использовать паттерн «стратегия» (Strategy) и фабрику моделей, что упрощает добавление новых моделей без изменения кода API.
Плохое качество прогнозов из-за неучтённых внешних факторов	Средняя	Высокое	Добавить больше внешних данных (погода, события), увеличить окно истории, использовать ансамбли. В крайнем случае – ручная калибровка.
Проблемы с производительностью инференса при большом количестве остановок (сотни)	Средняя	Среднее	Использовать батчевый инференс, кэширование (Redis), оптимизировать модели (ONNX, квантизация).
Нехватка времени на финальную презентацию	Низкая	Среднее	Света начинает готовить презентацию параллельно с разработкой (со 2-го дня), используя промежуточные результаты.

9. План коммуникации

- **Ежедневный стендап** (15 минут) в 10:00 (по московскому времени) – синхронизация статуса и блокеров.
 - **Общий чат** (Telegram/Slack) – для оперативных вопросов.
 - **Репозиторий** – все коммиты с осмысленными сообщениями, код-ревью через Pull Requests (каждый PR проверяет как минимум один участник).
 - **Документация** – ведётся в папке `/docs` в репозитории (Markdown).
-

10. Заключение

Данное ТЗ даёт чёткое понимание того, **что** нужно сделать, **кто** за что отвечает, и **в какие сроки**. Команда имеет все необходимые компетенции: сильный ML-инженер (Павел), опытный бэкенд и лид (Богуславский), креативный фронтенд-дизайнер (Баев) и аналитик-координатор (Света). Благодаря поэтапному плану и параллельной работе мы сможем за 7 дней создать рабочий прототип системы прогнозирования пассажиропотока, который не только покажет высокое качество прогнозов, но и впечатлит жюри красивым и удобным дашбордом.